
Curso de lenguaje C

Fundamentos de programación

Alumno:	Oliverio Landaverde Hernández
Grupo:	DSM1SM-26ERJ
Cuatrimestre:	1 (septiembre – diciembre 2026)
Profesora:	María de Monserrath Guerrero Chavero
Materia:	Fundamentos de programación
Tema:	Curso de lenguaje C
Lugar y fecha:	Jalpan de Serra, Querétaro, 6 de octubre de 2026

1 Estructura, variables y tipos [01]

```
#include <stdio.h> // librería E/S
int main()         // aquí empieza
{
    int edad = 20; // declarar y dar valor
    printf("%d\n", edad);
    return 0;      // 0 = terminó bien
}
```

Tipo	Guarda	Bytes	Formato
int	enteros: 7, -3	4	%d
float	reales: 3.14	4	%f
double	reales, más precisión	8	%lf (scanf)
char	un carácter: 'A'	1	%c
long	enteros grandes	4 u 8	%ld

- Variable = espacio de memoria con **nombre, tipo** y **valor**.
- Distingue mayúsculas: **Edad** $\neq$ **edad**.
- Cada sentencia termina en ; • los bloques van entre { }.
- Comentarios: // una línea • /* varias */
- sizeof(x) $\rightarrow$ bytes que ocupa x.

2 Entrada y salida estándar [02-04]

```
// mostrar
printf("Texto %d y %.2f\n", entero, real);
// leer: lleva &
scanf("%d", &entero);
// el espacio salta el Enter anterior
scanf(" %c", &letra);
```

Formato	Significado
%d	entero
%f / %.2f	real / real con 2 decimales
%5d	entero ocupando 5 espacios
%-5d	alineado a la izquierda
%c	carácter
%s	cadena de texto
%%	escribe el símbolo %
\n \t	salto de línea / tabulador

¡OJO! En scanf las variables llevan &, excepto las cadenas (char nombre[30]). El especificador es %d, no &d.

3 Operadores [02, 09, 10]

Tipo	Operadores	Notas
Aritméticos	+ - * / %	int/int da entero: $17/5 = 3$ · $17\%5 = 2$ (resto)
Relacionales	< > <= >= == !=	devuelven 1 (cierto) o 0 (falso)
Lógicos	&& !	Y, O, NO
Asignación	= += -= *= /= ++ --	$x += 3 = x = x + 3$

Prioridad (mayor → menor):

! → * / % → + - → < > <= >= → == != → && → || → =

- Decimales en una división: $a / (\text{float})b$ o $a / 2.0$
- $7 > 6 > 5$ vale 0: $(7 > 6) = 1$ y luego $1 > 5 = 0$.
- && solo es 1 si ambos son 1 · || es 1 si alguno es 1.

¡OJO! = asigna, == compara. if (x = 0) asigna 0 y la condición es siempre falsa.

4 Condicionales [11-19, 25]

```
if (condición)
    sentencia;
else if (otra)
    sentencia;
else
    sentencia;
```

```
switch (variable) {
    case 1:
        ...;
        break;
    case 2: case 3: // varios casos
        ...;
        break;
    default: // ninguno coincide
        ...;
}
```

- Más de una sentencia en el if → entre llaves { }.
- switch solo con **enteros o caracteres** (igualdad).
- Sin **break**, el switch sigue con los casos siguientes.
- Rango: $4 < V \ \&\& \ V < 9$ (nunca $4 < V < 9$).

¡OJO! Un ; justo después de if (x > 0);
termina el if: lo de abajo se ejecuta siempre.

5 Fórmulas que salen en los ejercicios

Qué	Fórmula
Ecuación 2.º grado [14]	$D = b^2 - 4ac$ · $D > 0$ dos soluciones, $D = 0$ una, $D < 0$ complejas · comprobar $a \neq 0$
MCD Euclides [13]	mientras $b \neq 0$: $r = a \% b$; $a = b$; $b = r$; $MCM = a \cdot b / MCD$
Bisiesto [18]	$(a \% 4 == 0 \ \&\& \ a \% 100 != 0) \ \ a \% 400 == 0$
Letra del NIF [19]	"TRWAGMYFPDXBNJZSQVHLCKE"[dni % 23]

6 Bucles [20-28]

```
while (condición) {  
    ...    // 0 o más veces  
}
```

```
do {  
    ...    // 1 o más veces  
} while (condición);
```

```
for (i = 0; i < n; i++) {  
    ...    // vueltas conocidas  
}
```

- **while**: no sabes cuántas vueltas y puede que ninguna.
- **do-while**: menús y validar datos (al menos una vez). Lleva ; al final.
- **for**: sabes cuántas vueltas (vectores, tablas).
- Acumulador: **suma** = 0 antes, **suma** += x dentro. Contador: **cont**++.
- Centinela: valor especial que termina la lectura (0, -1...). [20, 22]
- **break** sale del bucle · **continue** salta a la siguiente vuelta.

¡OJO! Bucle infinito si la condición nunca cambia. Inicializa siempre acumuladores y contadores.

7 Ficheros [06-08, 16, 18, 23-31]

5 pasos (siempre iguales):

```
FILE *f; // 1. declarar
f = fopen("datos.txt", "r"); // 2. abrir
if (f == NULL) { // 3. comprobar
    printf("Error");
    return 1;
}
while (fscanf(f, "%d", &x) == 1) { // 4. leer
    ...
}
fclose(f); // 5. cerrar
```

Modo	Uso
"r"	leer (el fichero debe existir)
"w"	escribir: crea o BORRA el contenido
"a"	añadir al final sin borrar

- Escribir: `fprintf(f, "%d\n", x);` · Leer: `fscanf(f, "%d", &x);`
- Línea completa: `fgets(cadena, tamaño, f)` (devuelve NULL al final).
- `fscanf` devuelve cuántos datos leyó: usar `== n` en el bucle.

¡OJO! Con "w" pierdes lo anterior. Para acumular registros usa "a".

8 Funciones [13, 26, 32-35]

```
// cabecera
tipo nombre(tipo p1, tipo p2)
{
    ...
    return valor;
}

float media(int a, int b, int c)
{
    return (a + b + c) / 3.0;
}

// void: no devuelve nada
void saludar() { printf("Hola"); }
```

- Divide el programa en partes pequeñas y reutilizables.
- **Por valor:** recibe una copia; cambiarla no afecta al original.
- **Por referencia** (puntero): `void f(int *x) { *x = 5; }` y se llama `f(&var)`. [39]
- Un **return** devuelve **un** valor; para varios, usa punteros.
- Traza: al llamar se salta a la función; al **return** se vuelve. [34]
- Si va después de main, declara antes el **prototipo**:
`float media(int, int, int);`

9 Vectores (arrays) [29-31, 33-40]

```
int v[5] = {6, -4, -2, 7, 2};  
// índices 0..4  
v[0] = 10; // primer elemento  
for (i = 0; i < 5; i++) // recorrido  
    suma += v[i];  
// se pasa el vector y su tamaño  
void f(int v[], int tam)
```

- Índices de **0 a tam-1**. **v[tam]** está fuera (error grave).
- Un vector pasado a una función **sí** se modifica (se pasa su dirección).
- Máx/mín: empezar con **max = v[0]** y comparar con el resto. [39]
- Búsqueda secuencial: recorrer hasta encontrar; devolver posición o -1. [40]
- Normalizar: $(v[i] - \text{min}) / (\text{max} - \text{min}) \rightarrow$ entre 0 y 1. Cuidado si $\text{max} == \text{min}$. [38]
- Producto vectorial $r = u \times v$: [35]
 $r[0] = u[1]v[2] - u[2]v[1]$
 $r[1] = u[2]v[0] - u[0]v[2]$
 $r[2] = u[0]v[1] - u[1]v[0]$

10 Memoria dinámica [41, 42, 56]

```
#include <stdlib.h>
// reservar n floats
float *v = (float *)
    malloc(n * sizeof(float));
if (v == NULL) { ... sin memoria ... }
v[i] = 3.5; // se usa como vector
// agrandar
v = (float *)
    realloc(v, nuevo * sizeof(float));
// liberar SIEMPRE
free(v);
```

- Cuando el tamaño se conoce **durante la ejecución** (usuario o fichero).
- Fichero de cantidad desconocida: duplicar la capacidad con `realloc`. [42]
- Cada `malloc` necesita su `free`; si no, hay **fuga de memoria**.

11 Ordenación (burbuja) [43-45]

Compara vecinos y los intercambia si están desordenados; en cada pasada el mayor «sube» al final.

```
for (i = 0; i < n - 1; i++)
    for (j = 0; j < n - 1 - i; j++)
        // > ascendente, < descendente
        if (v[j] > v[j + 1]) {
            aux = v[j];
            v[j] = v[j + 1];
            v[j + 1] = aux;
        }
```

- Intercambio siempre con una variable **auxiliar**.
- **Vectores paralelos** (matrícula + km): al intercambiar uno, intercambia también los demás.
- Cadenas: se intercambian con `strcpy` y se comparan con `strcmp`.

12 Cadenas de car.

```
char nombre[30]; // 29 letras + '\0'
// lee UNA palabra (sin &)
scanf("%29s", nombre);
// lee la línea entera
fgets(frase, sizeof(frase), stdin);
// quita el Enter que guarda fgets
frase[strlen(frase, "\n")] = '\0';
```

Cadena = vector de char que termina en '\0'.

	Qué hace
string.h	
strlen(s)	longitud (sin contar '\0')
strcpy(dest, orig)	copia (no vale dest = orig)
strcat(dest, orig)	añade orig al final de dest
strcmp(a, b)	0 iguales · <0 a antes · >0 a después
strchr(s, 'c')	busca un carácter
strstr(s, "sub")	busca una subcadena

ctype.h: isalpha isdigit isalnum toupper tolower [48, 52]

¡OJO! Las cadenas no se comparan con == ni se copian con =. Usa strcmp y strcpy.

13 Matrices [53-56]

```
int m[3][4]; // 3 filas, 4 columnas
for (i = 0; i < 3; i++) // filas
    for (j = 0; j < 4; j++) // columnas
        m[i][j] = 0;
```

```
// 5 cadenas de hasta 19 letras
char nombres[5][20];
```

- Por filas: i por fuera. Por columnas: j por fuera. [53]
- Suma: $C[i][j] = A[i][j] + B[i][j]$ (mismo tamaño). [54]
- Imagen PGM: matriz de grises 0..max. Negativo: max - pixel. [56]

14 Estructuras (struct) [57-61]

```
typedef struct {  
    int dia, mes, anio;  
} Fecha;  
  
Fecha hoy = {5, 10, 2026};  
hoy.mes = 11;
```

```
struct Alumno {  
    char nombre[30];  
    int edad;  
    float nota;  
};  
// vector de structs  
struct Alumno clase[30];  
clase[0].nota = 8.5;
```

- Agrupa datos de **distinto tipo** de una misma cosa.
- Campo: **variable.campo** · con puntero: **puntero->campo**.
- Se pueden asignar enteras y anidar (una Fecha dentro de un Empleado).
- Por valor para consultar; por puntero (&) para modificar. [60]

15 Resumen de una página

Quiero...	Uso...
Repetir n veces	for
Repetir hasta que el usuario pare	while / do-while con centinela
Elegir entre valores exactos	switch
Elegir por rangos	if / else if
Muchos datos del mismo tipo	vector
Una tabla	matriz
Datos de distinto tipo juntos	struct
No saber el tamaño hasta ejecutar	malloc / realloc
Que los datos duren al cerrar	ficheros
...	parámetros por puntero